SHOPSMART – GROCERY WEB APPLICATION

Full Stack Development Project Documentation

1. INTRODUCTION

Project Title: SHOPSMART – GROCERY WEB APPLICATION

Team Members:

Team Leader: Bokka Sai Durga Rao

- Led project planning and coordinated overall development activities.
- Managed system architecture and supervised final integration and submission.

Team Member: Guntapalli Gamya

- Designed and developed the frontend interface using HTML and CSS.
- Ensured responsive design and improved user experience.

Team Member: Rana Siva Kumar

- Implemented JavaScript logic for cart functionality and dynamic updates.
- Managed product data handling and total price calculation.

Team Member: Chinta Mohana Sowjanya

- Conducted functional testing and verified system performance.
- Prepared project documentation and maintained report structure.

Introduction:

ShopSmart is a frontend-based grocery web application designed to simulate the functionality of an e-commerce platform. The system enables users to browse grocery items, add selected products to a shopping cart, remove items, and dynamically calculate the total cost in real time. The application is developed using core web technologies such as HTML5, CSS3, and JavaScript (ES6). The primary focus of the system is to implement dynamic DOM manipulation, event-driven programming, and responsive UI design without backend integration.

2. PROJECT OVERVIEW

Purpose

ShopSmart is a full-stack grocery web application developed to simplify online grocery shopping. It enables users to browse products, add items to the cart, and perform secure checkout. An admin dashboard allows efficient product and user management.

Key Features

• User Registration and Login (JWT Authentication)

• Secure Password Hashing using bcrypt

• Product Listing and Search

• Shopping Cart Management

• Checkout System (Stripe Test Mode & Cash Option)

• Admin Dashboard for Product Management

• Responsive UI Design

• Protected Routes for Authorized Users

• MongoDB Database Integration

3. ARCHITECTURE

Frontend

Developed using Angular with Bootstrap 5 for responsive and modern UI.

Backend

Built using Node.js and Express.js following RESTful API architecture.

Database

MongoDB is used as the NoSQL database with Mongoose for schema modeling.

4. SETUP INSTRUCTIONS

Prerequisites

• Node.js (v16 or above)

• MongoDB (Local or Atlas)

• npm

Installation Steps

• Clone the repository.

• Navigate to the project directory.

• Install backend dependencies using 'npm install' inside the server folder.

• Install frontend dependencies using 'npm install' inside the client folder.

• Configure environment variables in the .env file (MONGO_URI, JWT_SECRET, STRIPE_SECRET_KEY).

5. FOLDER STRUCTURE

1. **Presentation Layer (Client):** The Angular frontend handles the user interface, state management, and routing. It communicates securely with the backend via RESTful APIs, attaching JWT tokens via HTTP Interceptors.

2. **Application Layer (Server):** The Express server receives requests, manages authorization via middleware guards, processes business logic (e.g. cart totals, order statuses), and interacts with the database.

6. RUNNING THE APPLICATION

Backend: Navigate to server folder and run 'npm start'.

Frontend: Navigate to client folder and run 'npm start'.

Backend runs on: http://localhost:5000

Frontend runs on: http://localhost:4200

7. API Documentation

User Routes

• POST /api/users/register – Register new user

• POST /api/users/login – Login user

• GET /api/users/profile – Get user profile (Protected)

Product Routes

• GET /api/products – Get all products

• GET /api/products/:id – Get single product

• POST /api/products – Add product (Admin)

• PUT /api/products/:id – Update product

• DELETE /api/products/:id – Delete product

Order Routes

• POST /api/orders – Create new order

• GET /api/orders – Get all orders (Admin)

8. AUTHENTICATION

Authentication is implemented using **JWT (JSON Web Tokens)**.

Process:

1. User logs in.
2. Server generates JWT token.
3. Token stored in browser (localStorage).
4. Protected routes verify token using middleware.

Authorization:

- Role-based access (Admin/User).
- Middleware checks user role before access..

9. USER INTERFACE

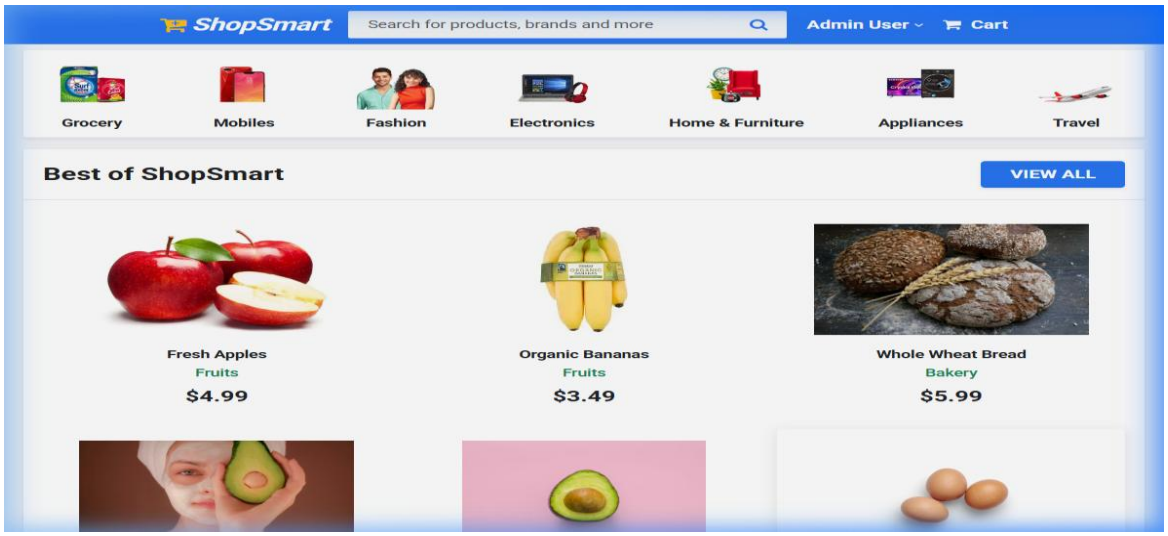

Fig 1: Homepage Layout

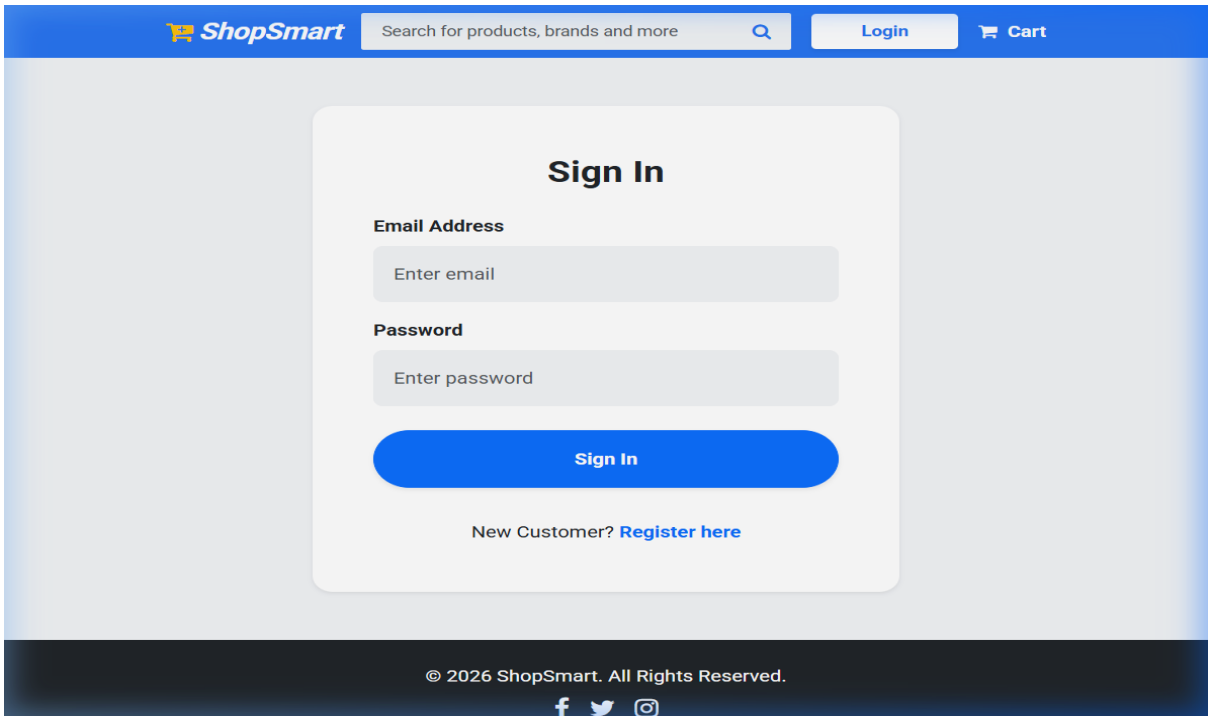

Fig 2: Sign In page

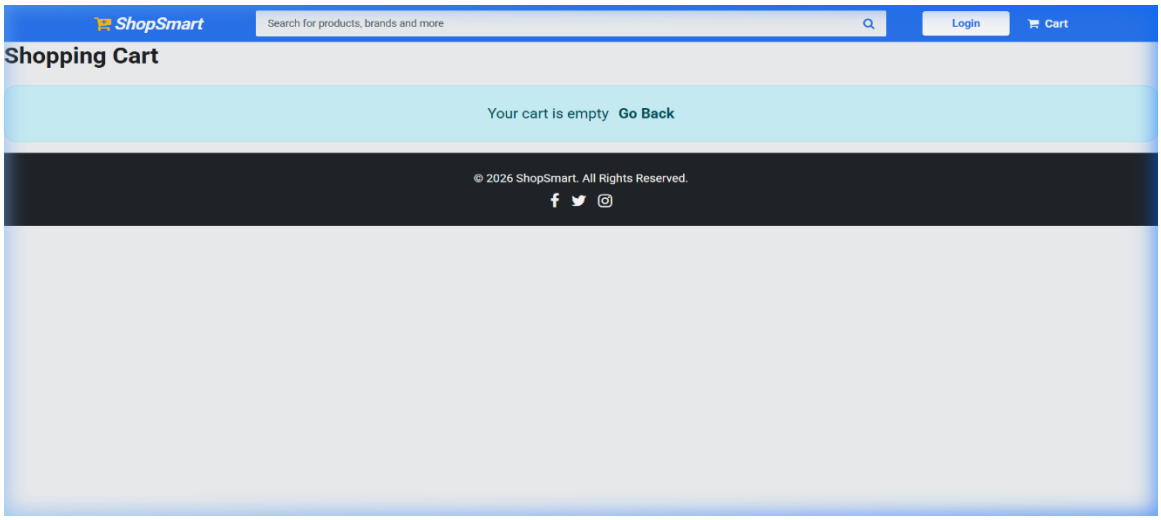

Fig 3: Cart with items page

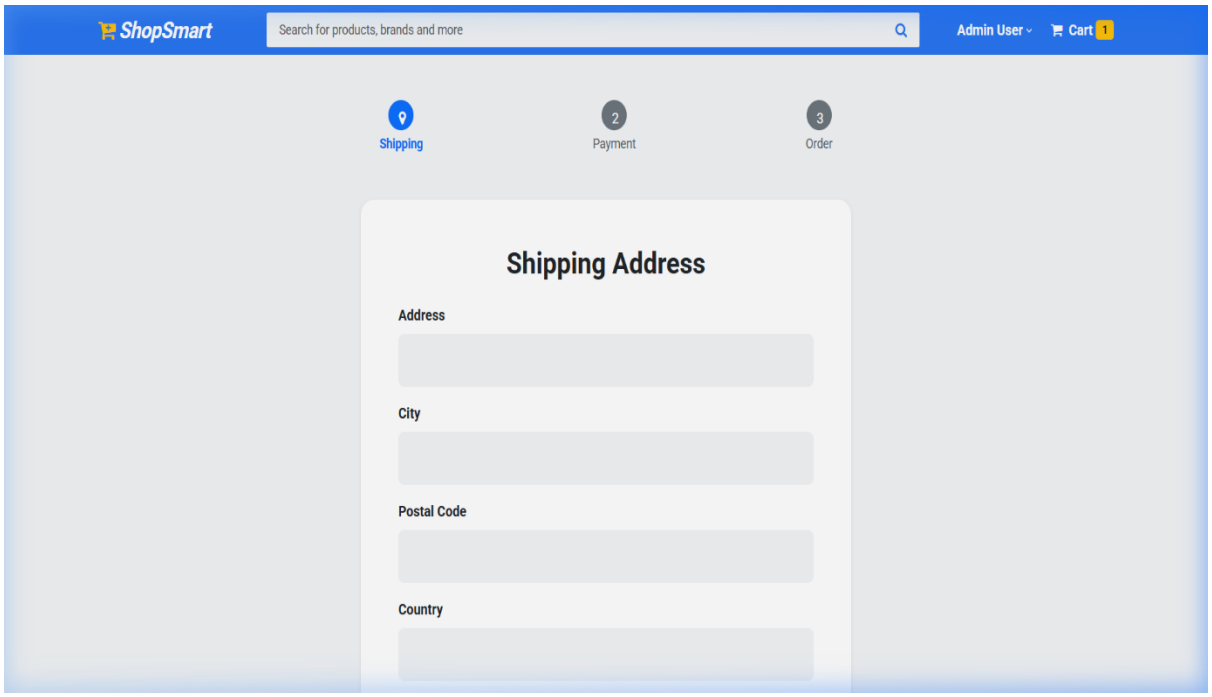

Fig 4: Shipping address filling page

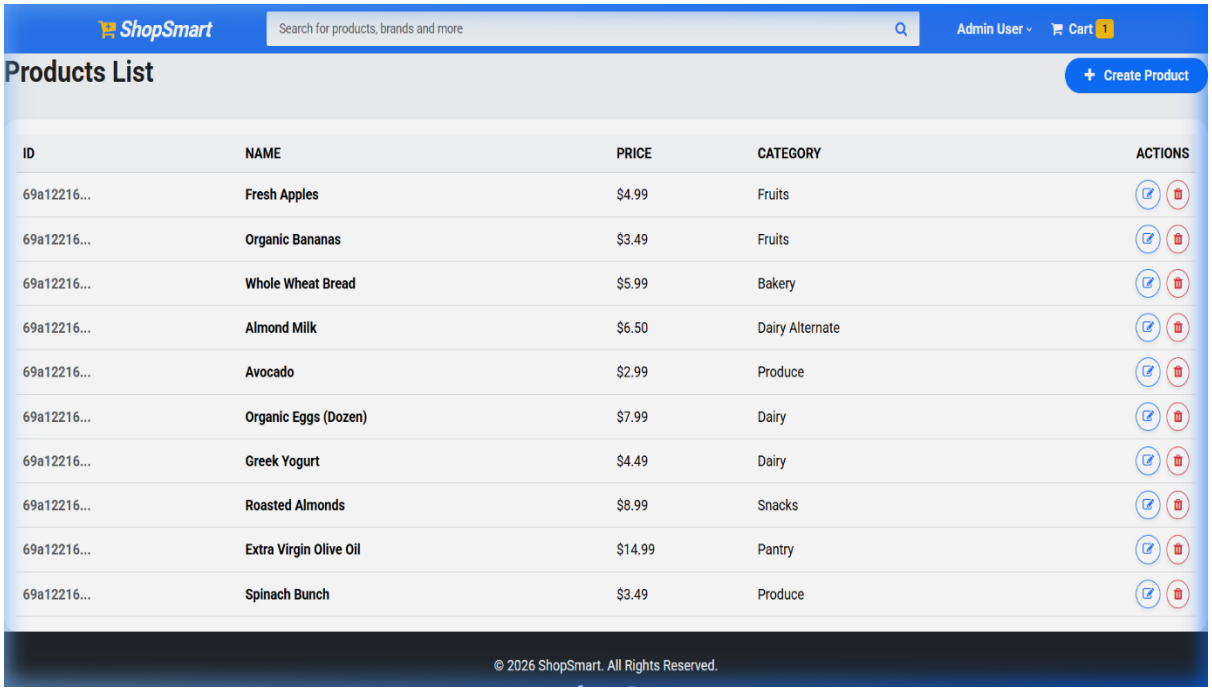

Fig 5: Product Listing and Total price calculation

10. TESTING

Manual testing was performed for authentication, product management, cart, and checkout features.

11. FUTURE ENHANCEMENTS

- Mobile application version
- Real-time payment gateway integration
- Email notifications
- Product recommendation system
- Order tracking feature
- Cloud deployment